A COMPUTER-ASSISTED LOWER BOUND FOR CUBIC CLAW-FREE COUNTEREXAMPLES TO THE ERDŐS–GYÁRFÁS CONJECTURE

[AUTHOR NAME]

Abstract. The Erdős–Gyárfás conjecture asserts that every graph of minimum degree at least three contains a cycle whose length is a power of two. Salehi Nowbandegani, Esfandiari, Shirdareh Haghighi and Bibak reduced the cubic claw-free case to the following question about simple cubic graphs H: must H contain a cycle of some length ℓ with $2\ell \le 2^k \le 3\ell$? They showed that a cubic claw-free counterexample has at least 114 vertices. Writing ν for the minimum order of a connected simple cubic graph with no cycle length in these "windows", the bound 114 is equivalent to $\nu \ge 38$. Using SAT modulo symmetries, we show by computer that $\nu \ge 40$, with the decisive case confirmed by two different implementations of the cycle-length constraint, and therefore that every cubic claw-free counterexample has at least 120 vertices. With a new, validated cycle-length propagator we further show $\nu \ge 58$, which would give the bound 174; since this second computation currently relies on a single implementation, we state it separately and provide a complete protocol for independent replication. As a by-product we verify the windows condition for all 111,360 connected cubic vertex-transitive graphs on at most 1280 vertices, with explicit cycle certificates, and we identify the unique graph in this census that has no cycle length in the windows below 17.

1. Introduction

For a graph G let $\mathcal{C}(G)$ denote the set of lengths of its cycles. Around 1995 Erdős and Gyárfás asked whether every graph of minimum degree at least three has a cycle whose length is a power of two; the question appeared in print in Erdős's 1997 problem paper [5], where he remarks that he and Gyárfás had by then come to believe the answer is negative, although they had not found a counterexample even for minimum degree three.

Conjecture 1.1 (Erdős–Gyárfás)**.** *If $\delta(G) \ge 3$, then $\mathcal{C}(G)$ contains a power of two.*

The conjecture is open. Liu and Montgomery [14] proved that it holds, in a strong form, for all graphs of sufficiently large *average* degree, refuting the stronger belief stated in [5]; their constant is not computed, and the case of minimum degree three is untouched by their methods. The conjecture is known for $K_{1,m}$-free graphs under degree conditions [20], planar claw-free graphs [3], 3-connected cubic planar graphs [9], P_8-free [7], P_{10}-free [11] and, with computer assistance, P_{13}-free graphs [10], and graphs of diameter two [1]. Computer searches show that counterexamples must be fairly large: Markström [15] showed that a cubic counterexample has at least 30 vertices, and very recently Ducoffe and Dumitru [4] verified the conjecture for all graphs on at most 40 vertices, all bipartite graphs on at most 66 vertices and all cubic graphs on at most 48 vertices, and also established structural properties of minimal counterexamples (see also Carr [2]). On the constructive side, Markström found cubic graphs on 24 vertices without cycles of length 4 or 8 [15], Exoo constructed a cubic graph on 78 vertices without cycles of length 4, 8 or 16 [6], and Ripatti showed by exhaustive search that 78 is the smallest possible order of such a graph [18].

Date: October 2026.

2020 *Mathematics Subject Classification.* 05C38 (primary); 05C85, 68R10 (secondary).

Key words and phrases. Erdős–Gyárfás conjecture, cycle lengths, cubic graphs, claw-free graphs, SAT modulo symmetries, computer-assisted proof.

The cubic claw-free case. Hobbs asked whether Conjecture 1.1 holds for claw-free graphs. Salehi Nowbandegani, Esfandiari, Shirdareh Haghighi and Bibak [19] studied this question and showed that cubic claw-free graphs without 4-cycles are exactly the truncations of simple cubic graphs. Since a cycle of length ℓ in a cubic graph H gives cycles of every length between 2ℓ and 3ℓ in its truncation, they proposed the following conjecture, which would settle the cubic claw-free case.

Conjecture 1.2 ([19, Conjecture 8])**.** *Every cubic graph contains a cycle of some length ℓ such that $2\ell \le 2^k < 3\ell$ for some integer k.*

They also proved by a counting argument that any cubic claw-free counterexample to Conjecture 1.1 has at least 114 vertices [19, Theorem 9]. As far as we are aware this has remained the best bound for the cubic claw-free class; for instance it is quoted in [11], and the general cubic bound of [4] only implies that such a counterexample has more than 48 vertices.

Our results. Let

$$\mathcal{W} \;=\; \bigcup_{k\ge 3} \mathcal{W}_k, \qquad \mathcal{W}_k = \{\ell \in \mathbb{Z} :\; 2^k/3 \le \ell \le 2^{k-1}\},$$

so that

$$\mathcal{W} = \{3,4\} \cup \{6,7,8\} \cup \{11,\dots,16\} \cup \{22,\dots,32\} \cup \{43,\dots,64\} \cup \{86,\dots,128\} \cup \cdots .$$

A positive integer $\ell \ge 3$ lies in $\mathcal{W}$ if and only if the interval $[2\ell, 3\ell]$ contains a power of two. Define

$$\nu \;=\; \min\{\, |V(H)| :\; H \text{ a connected simple cubic graph with } \mathcal{C}(H) \cap \mathcal{W} = \emptyset \,\},$$

with $\nu = \infty$ if no such graph exists; Conjecture 1.2 is the statement $\nu = \infty$. As we recall in Section 2, every cubic claw-free counterexample to Conjecture 1.1 has at least 3ν vertices, and the argument of [19, Theorem 9] shows $\nu \ge 38$.

Theorem 1.3. *$\nu \ge 40$. Consequently, every cubic claw-free counterexample to the Erdős–Gyárfás conjecture has at least 120 vertices.*

Theorem 1.3 is computer-assisted. The only order not covered by [19] is 38, and that case was settled by two different implementations of the cycle-length constraint inside the SAT modulo symmetries framework [13]; see Section 4.

Theorem 1.4 (computer-assisted, single implementation)**.** *Our computation shows $\nu \ge 58$. If correct, every cubic claw-free counterexample to the Erdős–Gyárfás conjecture has at least 174 vertices.*

The orders $40 \le n \le 56$ in Theorem 1.4 were decided with a new cycle-length propagator (Section 3), which we validated against known enumeration counts and against the alternative implementation wherever the latter finished. We nevertheless regard an independent replication as necessary before Theorem 1.4 is relied upon, and Section 5 gives a complete protocol for it; the author's own computational resources were insufficient to finish the slower alternative implementation beyond $n = 38$.

Finally, in Section 6 we check Conjecture 1.2 on the census of cubic vertex-transitive graphs [17].

Proposition 1.5. *Every connected cubic vertex-transitive graph on at most 1280 vertices has a cycle whose length lies in $\mathcal{W}$. Exactly one of these 111,360 graphs, which has 1080 vertices, has no cycle length in $\mathcal{W} \cap [3,16]$; its cycle lengths up to 34 are 5 and $18, 19, \dots, 34$.*

Every presence claim in Proposition 1.5 is backed by an explicit cycle that was checked by an independent verifier.

2. The reduction

All graphs are finite and simple. The *truncation* $T(H)$ of a cubic graph H is obtained by replacing every vertex v by a triangle $\{(v,e) : e \ni v\}$ and joining (v,e) to (w,e) for every edge $e = vw$. Then $T(H)$ is cubic and claw-free, has $3|V(H)|$ vertices, and every vertex lies in exactly one triangle.

Proposition 2.1 ([19, Proposition 6])**.** *The map $H \mapsto T(H)$ is a bijection between simple cubic graphs and simple cubic claw-free graphs that contain no 4-cycle. The inverse $G \mapsto \widehat{G}$ contracts the (vertex-disjoint) triangles of G.*

Lemma 2.2 (cf. [19, Corollary 7])**.** *Let H be cubic. If $\ell \in \mathcal{C}(H)$, then $\{2\ell, 2\ell+1, \dots, 3\ell\} \subseteq \mathcal{C}(T(H))$. Moreover*

$$\mathcal{C}(T(H)) \;=\; \{3\} \;\cup \bigcup_{\ell \in \mathcal{C}(H)} \{2\ell, \dots, 3\ell\}.$$

Proof. Let $C = v_1 v_2 \cdots v_\ell v_1$ be a cycle of H with edges $e_i = v_i v_{i+1}$ (indices mod ℓ). In $T(H)$ the edges $(v_i, e_i)(v_{i+1}, e_i)$ are present, and inside the triangle of v_i one can go from (v_i, e_{i-1}) to (v_i, e_i) either directly (one edge) or through the third vertex of that triangle (two edges). These choices are independent for different i and yield cycles of every length $\ell + \ell + j$ with $0 \le j \le \ell$. Conversely, a cycle of $T(H)$ that is not a triangle enters and leaves each triangle it meets exactly once, so contracting triangles maps it to a cycle of H of some length ℓ, and its own length is ℓ plus between ℓ and 2ℓ edges inside triangles. □

Only the first statement of Lemma 2.2 is needed below; the second explains why the windows $\mathcal{W}$ are exactly the right condition.

Lemma 2.3. *Let G be a cubic claw-free graph without a cycle whose length is a power of two. Then every component G' of G satisfies $G' = T(H)$ for a connected simple cubic graph H with $\mathcal{C}(H) \cap \mathcal{W} = \emptyset$. In particular $|V(G)| \ge |V(G')| = 3|V(H)| \ge 3\nu$.*

Proof. G' is cubic, claw-free and has no 4-cycle, so $G' = T(H)$ with $H = \widehat{G'}$ simple, cubic and connected by Proposition 2.1. If some $\ell \in \mathcal{C}(H)$ lay in $\mathcal{W}_k$, then $2\ell \le 2^k \le 3\ell$ and Lemma 2.2 would give a cycle of length 2^k in G'. □

Thus Theorem 1.3 follows from $\nu \ge 40$ and Theorem 1.4 from $\nu \ge 58$. Since a cubic graph has an even number of vertices, $\nu \ge 58$ is the statement that no connected simple cubic graph on $n \le 56$ vertices avoids $\mathcal{W}$. A graph on $n \le 56$ vertices has no cycle longer than $56 < 86$, so only the windows up to $\{43, \dots, 64\}$ matter in this range.

3. Computational method

3.1. **SAT modulo symmetries.** We use SAT modulo symmetries (SMS) by Kirchweger and Szeider [12, 13]. SMS solves a propositional encoding over edge variables e_{uv} $(0 \le u < v < n)$ with the CDCL solver CaDiCaL, extended by a propagator that performs complete dynamic symmetry breaking: a partial assignment is rejected whenever it cannot be completed to a graph that is lexicographically minimal among its isomorphic copies. Consequently, if SMS reports the encoding unsatisfiable, no graph on n vertices with the encoded properties exists, up to isomorphism. SMS further allows custom propagators that inspect the partially defined graph and return a set of edges whose simultaneous presence is forbidden; the solver then learns the corresponding clause.

3.2. **Encoding.** For each even n we generated, with the encoding builder shipped with SMS, the cardinality constraints that every vertex has degree exactly three (options `-d 3 -D 3`) and the compact encoding of girth at least five (option `-girth-compact 5`), which excludes cycles of length 3 and 4. Connectivity was enforced by the SMS option `-connected`; this loses nothing, because each component of a cubic graph avoiding $\mathcal{W}$ is again such a graph. The remaining lengths of $\mathcal{W} \cap [5, n]$ were excluded by one of the two propagators described next.

3.3. **Propagator A: a dedicated cycle-length propagator.** We implemented a new propagator `CycleLengthChecker` (listed in Appendix A), given a set F of forbidden lengths and a threshold s. Whenever it is called on a partially defined graph, it considers the graph P formed by the edges currently assigned *true* and searches for a cycle of P whose length lies in $F \cap [3, s]$; if the graph is fully defined it searches for a cycle with length in all of F. The search is a depth-first enumeration of paths starting at each vertex r and using only vertices larger than r, so that every cycle is found from its smallest vertex, with depth bounded by the largest admissible length. If a cycle of a forbidden length is found, its edge set is returned and SMS learns the clause $\bigvee_{e \in C} \neg x_e$.

Soundness. Every returned cycle consists of edges that are true in the current assignment, so it is present in every completion; the learned clause excludes only graphs that contain a forbidden cycle.

Completeness of the final check. Lengths above the threshold s are checked only on fully defined graphs. This is correct provided the propagator is consulted on every complete assignment before a graph is accepted. We confirmed this behaviour empirically: with $F = \{3, 4\}$ and $s = 3$ (so that 4-cycles are *only* detected on complete graphs), the enumeration of connected cubic graphs reproduces exactly the known numbers of girth at least five (Table 1).

We used $F = \{6, 7, 8\} \cup \{11, \dots, 16\} \cup \{22, \dots, 32\} \cup \{43, \dots, 64\}$ and $s = 16$. Since the graph under construction has maximum degree three, the short-cycle search costs $O(n \cdot 2^s)$ in the worst case and is negligible in practice.

3.4. **Propagator B: subgraph isomorphism.** SMS also provides a generic forbidden-subgraph propagator that calls the Glasgow Subgraph Solver [16] on the partially defined graph for each pattern in a list. We supplied the cycles C_ℓ for $\ell \in \{6, 7, 8\} \cup \{11, \dots, 16\} \cup \{22, \dots, 32\}$ as non-induced patterns (sufficient for $n \leq 42$). With the current version of the Glasgow solver this propagator aborted at run time, because SMS adds directed labelled edges to graphs it declares undirected; our patch passes the flag declaring these graphs directed and changes nothing else in the logic. Propagator B shares the SMS core and the encoding with Propagator A, but its cycle detection is independent of our new code.

3.5. **Validation.** Table 1 lists the checks performed before the production runs. The reference numbers are the numbers of connected cubic graphs of girth at least five and at least six (OEIS A014371 and A014372). In addition, Propagator B with $F = \{3, 4, 6, 7, 8\}$ (and no girth encoding) reported unsatisfiability for every even n from 20 to 36, independently confirming the bound $\nu \geq 38$ of [19] in the weaker form in which only the first two windows are excluded.

4. Results

For every even n with $20 \leq n \leq 56$ we asked SMS for a connected simple cubic graph on n vertices with $\mathcal{C} \cap \mathcal{W} = \emptyset$. All runs terminated with the answer *unsatisfiable* (exit code 20). Table 2 gives the running times on a single core of a cloud virtual machine.

Proof of Theorem 1.3. By [19, Theorem 9], $\nu \geq 38$; this is also confirmed by the runs with both propagators for $n \leq 36$. For $n = 38$, both Propagator A and Propagator B report that no

TABLE 1. Validation of the components used in the production runs. Each entry is the number of non-isomorphic connected cubic graphs enumerated with SMS; all agree with the known values.

n	10	12	14	16	18	20	known values
girth ≥ 5, Propagator A, $s = 16$	1	2	9	49	455		A014371
girth ≥ 5, Propagator A, $s = 3$	1	2	9	49	455		A014371
girth ≥ 5, Propagator B	1	2	9	49	455		A014371
girth ≥ 5, compact encoding only			9	49	455		A014371
girth ≥ 6, Propagator A			1	1	5	32	A014372
girth ≥ 6, Propagator B			1	1	5	32	A014372

TABLE 2. Running times (seconds) for deciding whether a connected simple cubic graph on n vertices with no cycle length in $\mathcal{W}$ exists. All completed runs returned *unsatisfiable*. A dash means that the run was not performed or not completed.

n	Prop. A	Prop. B	n	Prop. A	Prop. B
20	0.9	–	40	12	–
22	0.3	–	42	18	–
24	0.6	–	44	18	–
26	0.8	–	46	21	–
28	1.3	–	48	33	–
30	1.7	–	50	52	–
32	2.8	–	52	95	–
34	3.6	51	54	152	–
36	5.1	79	56	322	–
38	7	132			

connected simple cubic graph on 38 vertices avoids $\mathcal{W}$. Hence $\nu \geq 40$, and Lemma 2.3 gives the bound $3 \cdot 40 = 120$. □

Justification of Theorem 1.4. Propagator A reports unsatisfiability for all even n with $40 \leq n \leq 56$. Together with Theorem 1.3 this gives $\nu \geq 58$, and Lemma 2.3 gives $3 \cdot 58 = 174$. □

For $n \geq 40$, Propagator B is much slower: a cube-and-conquer run for $n = 40$ had completed 74 of 506 subproblems, with individual subproblems taking up to about three minutes, when it had to be stopped. At the top of the range the running time of Propagator A grows by a factor of about 1.6 to 2.1 per step of two vertices.

5. Protocol for independent replication

We invite readers with access to multi-core machines to replicate Theorem 1.4. All files mentioned are in the accompanying data package.

5.1. **Software.**

(1) SAT modulo symmetries, `https://github.com/markirch/sat-modulo-symmetries`, commit `63958bd09a871e484c59270a1d0f22d482dc5770`, with submodule `cadical_sms` at commit `b023aaf059babf867a7fdfc5fb342d52ffbccb25`.

(2) Glasgow Subgraph Solver, `https://github.com/ciaranm/glasgow-subgraph-solver`, commit `b62cc77777467e9fe19d9b558d706adb92989130`.
(3) Dependencies (Ubuntu 24.04): `cmake`, `g++`, `libboost-all-dev`, `libgmp-dev`.

Apply the file `sms_patch.diff` to the SMS source tree and place `cycleLengths.hpp` in the directory `src/graphPropagators/`. The patch (i) links GMP (needed by the Glasgow build), (ii) passes the "directed" flag for the labelled Glasgow graphs, and (iii) registers Propagator A with the options `-forbidden-cycle-lengths` and `-short-cycle-max`. Then run `./build-and-install.sh -s` and `pip install .` in the SMS directory.

5.2. **Run with Propagator A.** For each even $n \in \{38, \dots, 56\}$:

```
python3 -m pysms.graph_builder -v $n -D 3 -d 3 --girth-compact 5 \
       --no-solve --cnf-file enc_$n.cnf
smsg --vertices $n --connected --dimacs enc_$n.cnf \
     --forbidden-cycle-lengths 6-8,11-16,22-32,43-64 \
     --frequency-forbidden-subgraphs 1
echo $? # expected: 20 (unsatisfiable), and no graph printed
```

The longest run took 322 seconds on one core.

5.3. **Run with Propagator B (independent cycle detection).** Replace the option `-forbidden-cycle-lengths` and its argument by

`-forbidden-subgraph-file fW.txt`,

where `fW.txt` lists the cycles C_ℓ for $\ell \in \{6, 7, 8, 11, \dots, 16, 22, \dots, 32\}$ (add $43, \dots, 56$ for $n \geq 44$), one per line, each line consisting of ℓ followed by the edge list $0\,1,\ 1\,2,\ \dots,\ (\ell{-}1)\,0$. For $n \geq 40$ this needs cube-and-conquer and many cores. Cubes are produced with

```
smsg ... --simple-assignment-cutoff 20 > cubes.txt # must exit with 20
grep '^a ' cubes.txt > cubes.icnf
```

The exit code 20 certifies that the cubes cover the whole search space. Each range of cubes is then solved independently with `-cube-file cubes.icnf -cubes-range` a b. **Caution:** cube lines are numbered from 1 and both ends of the range are inclusive; a range starting at 0 is silently skipped by the version used here. The script `cnc.py` in the data package splits hard cubes recursively and logs every step.

5.4. **A fully independent check.** Propagators A and B share the SMS framework and the degree and girth encodings. A check that shares no code with ours would be valuable, for example an orderly generation of cubic graphs of girth at least five in the style of Brinkmann's generator, with pruning on the short windows, or a SAT encoding written from scratch in which the cycles of length 6–8 and 11–16 are excluded explicitly and longer cycles lazily. Since the conditions $\mathcal{C}(H) \cap \{3, 4, 6, 7, 8\} = \emptyset$ already force a very sparse local structure (girth five and pentagons pairwise far apart, or girth at least nine), such a search may well be feasible for $n \leq 56$.

6. Vertex-transitive cubic graphs

We used the census of Potočnik, Spiga and Verret [17] of all 111,360 connected cubic vertex-transitive graphs on at most 1280 vertices, in the sparse6 encoding distributed in the public repository `https://github.com/kguo-sagecode/cubic-vertextransitive-graphs`. In a vertex-transitive graph a cycle of length ℓ exists if and only if one exists through a fixed vertex, so for each graph and each target length we ran a depth-first search for cycles through vertex 0, pruned by distances to vertex 0.

Windows. For each graph we searched the lengths of $\mathcal{W} \cap [3, 64]$ in increasing order and recorded the first cycle found. Every graph has such a cycle. The first length found was 3 (160 graphs), 4 (5754), 6 (58,677), 7 (192), 8 (13,542), 11 (117), 12 (27,447), 13 (57), 14 (5350), 15 (31), 16 (32), and 22 for exactly one graph, denoted H_{1080}, on 1080 vertices. Each of the 111,360 recorded cycles was checked by an independent script (using NetworkX) for having distinct vertices, consecutive vertices adjacent and length in $\mathcal{W}$. For H_{1080} the cycle lengths through a vertex, computed up to 34, are 5 and $18, \ldots, 34$, and the absence of other cycle lengths up to 12 through vertex 0, and up to 10 through three further vertices, was confirmed independently with NetworkX. This proves Proposition 1.5. By Lemma 2.2, the truncation of H_{1080} (on 3240 vertices) has no cycle of length 4, 8, 16 or 32, but it has a cycle of length 64.

Powers of two. We also determined, for each census graph, the smallest power of two that is a cycle length: 4 for 5756 graphs, 8 for 16,624, 16 for 88,880, and 32 for the remaining 100. For these 100 graphs, an independent NetworkX computation confirmed the absence of cycles of length 4, 8 and 16, and explicit 32-cycles were found and checked. Their orders range from 180 to 1260; 53 of them have girth three, and each of these is the truncation of a cubic graph, in accordance with Lemma 2.2. Census computations of this kind for the Erdős–Gyárfás conjecture itself were also carried out recently by Gebendorfer, who in addition proved that every cubic bipartite vertex-transitive graph of girth six has a cycle of length 8, 16 or 32 [8]; our counts are consistent with his: exactly 14 of our 100 graphs are bipartite of girth six, matching the 14 extremal graphs he reports in that class.

7. Concluding remarks

(1) The windows $\mathcal{W}_k$ have relative width 3/2 and the gaps between consecutive windows have relative width 4/3. A cubic graph avoids $\mathcal{W}$ only if its cycle spectrum skips every window, while the graph H_{1080} illustrates the typical failure: after a large gap, its cycle lengths form an unbroken run, and any run of length about one third of its starting value meets a window.
(2) The cost of our search grows by a factor of about two per two vertices near $n = 56$. Bounds well beyond $\nu \geq 58$, and hence beyond 174 for the cubic claw-free class, should be within reach of moderate parallel computation.
(3) Conjecture 1.2 is equivalent to $\nu = \infty$ and implies the Erdős–Gyárfás conjecture for cubic claw-free graphs. It may be a useful testing ground, since its conditions concern cycles in a smaller graph and every cubic graph is a candidate.

Data availability

The patch, the propagator, the forbidden-pattern files, the run logs, the cube-and-conquer driver and the census certificates are available at [repository URL to be added].

Use of AI tools

The literature search, the software development, the computations and the first draft of this manuscript were carried out with substantial assistance from Claude, an AI model developed by Anthropic. The author has checked the arguments, reproduced the computations reported here [to be confirmed by the author], and takes full responsibility for the content.

Acknowledgements

[To be added by the author.]

Appendix A. Source of Propagator A

```
// Forbid cycles of given lengths (non-induced). Lengths <= shortMax are checked on every
// (partially defined) call; longer lengths are checked only once the graph is fully defined.
#ifndef CYCLE_LENGTHS_HPP
#define CYCLE_LENGTHS_HPP
#include "../graphChecker.hpp"
#include <sstream>

class CycleLengthChecker : public PartiallyDefinedGraphChecker
{
    std::vector<char> forb; int maxLen = 0; int shortMax;
    int n; std::vector<std::vector<int>> adj; std::vector<int> path; std::vector<char> vis;
    int start, curLimit;

    bool dfs(int u) {
        int len = (int)path.size(); // vertices on path = edges used + 1
        for (int v : adj[u]) {
            if (v == start && len >= 3 && forb[len] && len <= curLimit) {
                forbidden_graph_t g;
                for (int i = 0; i < len; i++) g.push_back({truth_value_true, {path[i], path[(i +
                     1) % len]}});
                throw g;
            }
            if (v <= start || vis[v] || len >= curLimit) continue;
            vis[v] = 1; path.push_back(v);
            dfs(v);
            path.pop_back(); vis[v] = 0;
        }
        return false;
    }

public:
    CycleLengthChecker(int frequency, const std::string &spec, int shortMax) :
         PartiallyDefinedGraphChecker(frequency), shortMax(shortMax) {
        std::stringstream ss(spec); std::string tok; std::vector<int> L;
        while (std::getline(ss, tok, ',')) {
            auto d = tok.find('-');
            if (d == std::string::npos) L.push_back(std::stoi(tok));
            else for (int x = std::stoi(tok.substr(0, d)); x <= std::stoi(tok.substr(d + 1)); x
                 ++) L.push_back(x);
        }
        for (int x : L) maxLen = std::max(maxLen, x);
        forb.assign(maxLen + 2, 0); for (int x : L) forb[x] = 1;
    }

    void checkProperty(const adjacency_matrix_t &matrix) {
        n = (int)matrix.size();
        adj.assign(n, {}); bool full = true;
        for (int i = 0; i < n; i++) for (int j = 0; j < n; j++) if (i != j) {
            if (matrix[i][j] == truth_value_true) adj[i].push_back(j);
            else if (matrix[i][j] == truth_value_unknown) full = false;
        }
        curLimit = full ? maxLen : std::min(shortMax, maxLen);
        vis.assign(n, 0);
        for (start = 0; start < n; start++) {
            path.assign(1, start); vis[start] = 1;
```

```
            dfs(start);
            vis[start] = 0;
        }
    }
};
#endif
```

References

[1] A. Carr, *Cycles of length 4 or 8 in graphs with diameter 2 and minimum degree at least 3*, Bull. Inst. Combin. Appl., to appear; arXiv:2508.19302.

[2] A. Carr, *Every minimal counterexample to the Erdős–Gyárfás conjecture is predominantly cubic*, arXiv:2605.22844, 2026.

[3] D. Daniel and S. E. Shauger, *A result on the Erdős–Gyárfás conjecture in planar graphs*, Congr. Numer. **153** (2001), 129–140.

[4] G. Ducoffe and B. Dumitru, *Towards a more structured search for Erdős–Gyárfás counter-examples*, arXiv:2609.28594, 2026.

[5] P. Erdős, *Some old and new problems in various branches of combinatorics*, Discrete Math. **165/166** (1997), 227–231.

[6] G. Exoo, *Three graphs and the Erdős–Gyárfás conjecture*, arXiv:1403.5636, 2014.

[7] Y. Gao and S. Shan, *Erdős–Gyárfás conjecture for P_8-free graphs*, Graphs Combin. **38** (2022), Article 168.

[8] J. J. Gebendorfer, preprint on the Erdős–Gyárfás conjecture for cubic bipartite vertex-transitive graphs of girth six, Zenodo, 2026, doi:10.5281/zenodo.18407880.

[9] C. C. Heckman and R. Krakovski, *Erdős–Gyárfás conjecture for cubic planar graphs*, Electron. J. Combin. **20**(2) (2013), #P7.

[10] A. S. Hegde, R. B. Sandeep and P. Shashank, *Erdős–Gyárfás conjecture on graphs without long induced paths*, arXiv:2410.22842, 2024.

[11] Z. Hu and C. Shen, *The Erdős–Gyárfás conjecture holds for P_{10}-free graphs*, Discrete Math. **347** (2024), 114175.

[12] M. Kirchweger and S. Szeider, *SAT modulo symmetries for graph generation*, in: 27th International Conference on Principles and Practice of Constraint Programming (CP 2021), LIPIcs **210** (2021), 34:1–34:16.

[13] M. Kirchweger and S. Szeider, *SAT modulo symmetries for graph generation and enumeration*, ACM Trans. Comput. Log. **25**(3) (2024), Article 18.

[14] H. Liu and R. Montgomery, *A solution to Erdős and Hajnal's odd cycle problem*, J. Amer. Math. Soc. **36** (2023), 1191–1234.

[15] K. Markström, *Extremal graphs for some problems on cycles in graphs*, Congr. Numer. **171** (2004), 179–192.

[16] C. McCreesh, P. Prosser and J. Trimble, *The Glasgow Subgraph Solver: using constraint programming to tackle hard subgraph isomorphism problem variants*, in: Graph Transformation (ICGT 2020), Lecture Notes in Comput. Sci. **12150**, Springer, 2020, 316–324.

[17] P. Potočnik, P. Spiga and G. Verret, *Cubic vertex-transitive graphs on up to 1280 vertices*, J. Symbolic Comput. **50** (2013), 465–477.

[18] A. V. Ripatti, *Faster approach for exhaustive search over graphs with certain properties*, conference contribution, BAIKAL 2017, `https://isem.irk.ru/conferences/mopt2017/Abstracts/BAIKAL_2017_paper_138.pdf`.

[19] P. Salehi Nowbandegani, H. Esfandiari, M. H. Shirdareh Haghighi and K. Bibak, *On the Erdős–Gyárfás conjecture in claw-free graphs*, Discuss. Math. Graph Theory **34** (2014), 635–640.

[20] S. E. Shauger, *Results on the Erdős–Gyárfás conjecture in $K_{1,m}$-free graphs*, Congr. Numer. **134** (1998), 61–65.

[Affiliation, City, Country]
Email address: `[email address]`